



Instituto Superior Politécnico de Tecnologias e Ciências (ISPTEC)
Departamento de Engenharia e Tecnologias

Base de dados II

Licenciatura em Engenharia Informática

2025/2026

LAB 02

Docente:
Judson Paiva

INTEGRANTES DO GRUPO 2

Jodelfi Marimba - 20230973

Hélcio da Silva - 20230038

Marinácio Hulu – 20221788

Elineu Tomás - 20230812

PARTE 1 - RESULTADOS

REGISTO DE RESULTADOS — FASE-1A	
Versão do Docker instalada	28.3.3
Versão do Docker Compose	V5.1.4
Sistema Operativo usado	Windows

Fase 1B – Criação e Inicialização do Cluster

REGISTO DE RESULTADOS — FASE-1B	
N.º de contentores com STATUS Up	4
Resultado de COUNT(*) FROM VENDA (Luanda)	30.000
Resultado de COUNT(*) FROM CLIENTE (Luanda)	5.000
Os dados do SQL foram carregados automaticamente? (S/N)	S

2.3 — Explorar os Fragmentos no MercadoKwanza

REGISTO DE RESULTADOS — FASE-2	
Total vendas — fragmento Luanda	10014
Total vendas — fragmento Benguela	10006
Total vendas — fragmento Huambo	9980
Soma dos 3 fragmentos = total? (S/N)	S
Query de fragmentação vertical escrita pelo grupo? (S/N)	S

REGISTO DE RESULTADOS — FASE-3	
Slave_IO_Running — Benguela	YES
Slave_SQL_Running — Benguela	YES
Seconds_Behind_Master — Benguela	0
Slave_IO_Running — Huambo	YES
Seconds_Behind_Master — Huambo	0

Os 3 produtos inseridos aparecem no Slave Benguela? (S/N)	S
Os 3 produtos inseridos aparecem no Slave Huambo? (S/N)	S

REGISTO DE RESULTADOS — FASE-3B	
O que aconteceu às leituras no Slave quando o Master estava pausado?	Retornou a quantidade total de elementos da tabela PRODUTO
O que aconteceu quando tentaram escrever com o Master pausado?	Dá um erro a dizer que o container está pausado
Após restaurar o Master, o Slave sincronizou automaticamente? (S/N)	S

REGISTO DE RESULTADOS — FASE 4A	
Quantidade ANTES do UPDATE	206
Quantidade APÓS UPDATE (antes do ROLLBACK)	201
Quantidade APÓS ROLLBACK (igual ao inicial?)	206
Quantidade APÓS COMMIT	201

PARTE 1 - REFLEXÃO ESCRITA

Fase 1

1. O que significa o mapeamento de portas '3307:3306'? Porque é que os nós Benguela e Huambo não podem também usar a porta 3306 no anfitrião?

R: O mapeamento de portas serve para expor as portas do container para o mundo exterior, associando-as à portas físicas. Não podem utilizar a mesma porta porque os sistemas operativos não permitem que uma porta seja escutada por mais do que um processo simultaneamente.

2. **Qual a vantagem de usar o volume './dados:/docker-entrypoint-initdb.d'? O que aconteceria se não usassem esse volume e quisessem carregar os dados manualmente?**

R: A vantagem de utilização do volume './dados:/docker-entrypoint-initdb.d' está na execução automática dos scripts, permitindo inicializar o container sempre que for iniciado. Se não se fizesse a utilização do volume, não teríamos dados carregados automaticamente e teríamos que importar em cada container individualmente.

3. **Se desligarem o computador e voltarem a ligar, os dados permanecem? Testem e documentem o que observam.**

R: Sim, permanecem.

Fase 2

1. **Segundo Özsu & Valduriez (2020), um esquema de fragmentação válido deve respeitar três propriedades: completude, disjunção e reconstituição. As views que criaram respeitam estas três propriedades? Justifiquem para cada uma.**

R: Sim.

Completude: nenhum dado da relação fragmentada foi perdido;

Disjunção: os fragmentos não possuem dados repetidos, excepto a chave candidata;

Reconstituição: é totalmente possível reconstruir a relação original através da união dos dois fragmentos.

2. **Quais são as limitações de simular fragmentação com VIEWS em vez de distribuir fisicamente os dados por diferentes servidores? O que é que as VIEWS não conseguem simular?**

R: Não é possível tirar proveito das vantagens da fragmentação porque as views ficam todas no mesmo local e, se aquele servidor cair, perde-se o acesso aos dados. Também não se conseguem observar

os impactos da latência e desempenho porque rodam no mesmo hardware.

3. **Num cenário real, que factores decidem se uma tabela deve ser fragmentada horizontalmente, verticalmente, ou não fragmentada de todo?**

R: O tipo de fragmentação depende principalmente de como os dados são utilizados. Se diferentes grupos utilizarem registos diferentes, implementa-se a fragmentação horizontal e,

se utilizarem atributos diferentes, implementa-se a fragmentação vertical. Não deve se implementar nenhum tipo de fragmentação se:

- a. Os pontos anteriores não se verificarem;
- b. A base de dados for muito pequena;
- c. Não melhorar o desempenho;
- d. Aumentar a complexidade das queries;

Fase 3

- 1. A replicação Master-Slave é síncrona ou assíncrona? O que significa isso para a consistência dos dados? Se um cliente comprar um produto em Luanda (Master) e imediatamente consultar o stock em Benguela (Slave), pode ver um valor desactualizado?**

R: Por padrão é assíncrona. Há grandes chances de se ter inconsistência nos dados porque o master confirma que a operação de escrita foi realizada antes de enviar os dados aos slaves e, se a comunicação falhar durante o envio, todos os dados que não foram enviados são perdidos. Pode ver um valor desactualizado se o tempo de envio for maior que o tempo gasto para fazer a operação de consulta no outro stock.

- 2. Qual o papel do binary log (binlog) na replicação? O que aconteceria se o binary log fosse apagado acidentalmente no Master enquanto os Slaves ainda não tinham lido todas as entradas?**

R: O binary log serve para armazenar os eventos realizados no master e é enviado aos slaves para que eles possam requisitar ao master os eventos que ainda não recebeu. Se o binlog fosse apagado por acidente, haveria inconsistência de dados porque os slaves não teriam as últimas actualizações do master e o processo de replicação falharia.

- 3. Numa arquitectura real com 10 Slaves, o que acontece se o Master cair permanentemente? Que estratégia poderia o grupo adoptar para tornar o sistema mais resiliente?**

R: Se o master cai permanentemente as operações de escrita ficam bloqueadas e o sistema distribuído servirá apenas para leitura de dados, como visto no caso anterior. Para deixar o sistema mais resiliente podemos implementar o processo de failover que permitirá eleger um novo master para lidar com as operações de

escrita. Este processo pode ser realizado pelo administrador de rede de forma manual ou configurar o sistema para automaticamente realizar o procedimento definido para realização do failover.

Fase 4

- 1. O protocolo Two-Phase Commit que implementaram é simplificado — não trata todos os casos de falha. O que aconteceria se o nó Benguela fizesse COMMIT mas a ligação falhasse antes de o nó Luanda fazer o seu COMMIT? Este estado chama-se 'in doubt transaction'. Como poderia ser resolvido?**

R: Se Benguela fizer COMMIT e a ligação falhar antes de Luanda confirmar, os nós ficam inconsistentes: Benguela guarda a alteração e Luanda pode não registar a venda. Isto é uma in-doubt transaction.

Num Two-Phase Commit completo, o problema resolve-se com um coordenador que guarda a decisão final num log persistente. Quando a ligação volta, os nós consultam essa decisão e terminam a transação da mesma forma.

- 2. Qual a diferença entre o nível de isolamento READ COMMITTED e SERIALIZABLE? Para um sistema de stock em tempo real, qual escolheriam e porquê?**

R: READ COMMITTED só lê dados já confirmados, mas pode haver alterações entre leituras da mesma transação. SERIALIZABLE é mais restritivo e faz as transações parecerem executadas uma de cada vez, evitando conflitos.

Num sistema de stock em tempo real, eu escolheria SERIALIZABLE se a prioridade fosse máxima consistência. Se fosse preciso melhor desempenho, READ COMMITTED com bloqueios explícitos, como SELECT ... FOR UPDATE, pode ser uma boa opção.

- 3. O uso de 'FOR UPDATE' no SELECT em Benguela bloqueou o registo. Que problema resolve este bloqueio? Que problema pode criar se muitas transações tentarem aceder ao mesmo produto em simultâneo?**

R: FOR UPDATE bloqueia a linha até a transação terminar, evitando que outra transação a altere ao mesmo tempo. Assim, reduz erros de concorrência e evita vendas com o mesmo stock.

Se muitas transações tentarem aceder ao mesmo produto, podem surgir esperas longas e até deadlocks, obrigando o sistema a cancelar uma das transações.

PARTE 2 – IMPLEMENTAÇÃO

No Cenário B foi simulado um período de promoções no MercadoKwanza, em que a sede de Luanda (Master) registou 100 novos clientes. Os clientes foram distribuídos aproximadamente por 50% em Luanda, 30% em Benguela e 20% no Huambo.

Inicialmente, foi registado o número de clientes existentes em todos os nós (Master e Slaves). De seguida, executou-se um script SQL/Python para inserir os 100 novos clientes no Master. Após a inserção, verificou-se a propagação dos dados para os Slaves através da consulta COUNT(*) e da observação do campo Seconds_Behind_Master com o comando SHOW SLAVE STATUS.

Posteriormente, foram criadas três views para simular a fragmentação horizontal da tabela CLIENTE, dividindo os registos por província.

```
CREATE OR REPLACE VIEW frag_cliente_luanda AS
SELECT * FROM CLIENTE
WHERE provincia_id = 1;
```

```
CREATE OR REPLACE VIEW frag_cliente_benguela AS
SELECT * FROM CLIENTE
WHERE provincia_id = 2;
```

```
CREATE OR REPLACE VIEW frag_cliente_huambo AS
SELECT * FROM CLIENTE
WHERE provincia_id = 3;
```

Para verificar a propriedade de completude da fragmentação, foi comparada a soma dos registos existentes nas três views com o número total de clientes.

```
SELECT
  (SELECT COUNT(*) FROM frag_cliente_luanda) +
  (SELECT COUNT(*) FROM frag_cliente_benguela) +
  (SELECT COUNT(*) FROM frag_cliente_huambo) AS soma_fragmentos,
  (SELECT COUNT(*) FROM CLIENTE) AS total_geral;
```

Por fim, foi calculado o faturamento médio por cliente em cada província, permitindo avaliar a importância de cada fragmento para o negócio.

```
SELECT p.nome AS provincia,  
       COUNT(DISTINCT c.id) AS clientes,  
       ROUND(SUM(v.total)/COUNT(DISTINCT c.id),2) AS  
faturamento_medio_por_cliente  
FROM CLIENTE c  
JOIN PROVINCIA p ON c.provincia_id = p.id  
LEFT JOIN VENDA v ON v.cliente_id = c.id  
GROUP BY p.nome  
ORDER BY faturamento_medio_por_cliente DESC;
```

PARTE 2 – ANÁLISE

REGISTO DE RESULTADOS — FASE-3B	
COUNT CLIENTE antes dos INSERTs (Luanda)	5000
COUNT CLIENTE após INSERTs (Luanda)	5100
COUNT CLIENTE após 5s (Benguela — propagou?)	5100
Soma dos 3 fragmentos = total geral? (S/N)	S
Província com maior faturamento médio por cliente	Luanda
Seconds_Behind_Master máximo observado	0

GITHUB

LINK: <https://github.com/Jodelf1/bdd2-lab02-grupo2.git>

CONCLUSÃO

Ao longo deste laboratório foi possível compreender, na prática, como funcionam as bases de dados distribuídas e quais as vantagens da distribuição dos dados por vários servidores. Verificou-se que a fragmentação permite organizar a informação de forma mais eficiente, aproximando os dados dos utilizadores que mais os utilizam, enquanto a replicação aumenta a disponibilidade e garante que existem cópias dos dados em diferentes nós.

Também foi possível observar que a replicação não é instantânea. Existe um pequeno intervalo entre a escrita no servidor principal e a atualização dos servidores secundários, o que significa que, durante alguns segundos, diferentes nós podem apresentar informações diferentes.

Além disso, percebeu-se a importância das transações para garantir que operações relacionadas são executadas corretamente, evitando que a base de dados fique num estado inconsistente quando ocorre uma falha. O uso de bloqueios e do protocolo Two-Phase Commit mostrou como os sistemas distribuídos procuram manter a integridade dos dados, mesmo quando várias operações acontecem ao mesmo tempo.

Em conclusão, este laboratório permitiu compreender não apenas os conceitos teóricos, mas também a forma como estes são aplicados em sistemas reais. A realização das experiências tornou mais evidente a importância da replicação, da fragmentação e das transações para construir aplicações distribuídas mais fiáveis, consistentes e preparadas para lidar com falhas.

REFERÊNCIAS BIBLIOGRÁFICAS

- MySQL 8.0 Reference Manual. Oracle Corporatio
- GILBERT, S.; LYNCH, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, 33(2). — Prova formal do Teorema CAP.
- GRAY, J.; LAMPORT, L. (2006). Consensus on Transaction Commit. ACM Transactions on Database Systems, 31(1), pp. 133–160. — Two-Phase Commit e Paxos.
- KLEPPMANN, M. (2017). Designing Data-Intensive Applications. O'Reilly Media. — Capítulos 5, 6 e 9: replicação, particionamento e transações distribuídas.
- CHODOROW, K. (2019). MongoDB: The Definitive Guide. 3.ª ed. O'Reilly Media. — Modelo de documentos, sharding e agregações.
- COULOURIS, G. et al. (2012). Distributed Systems: Concepts and Design. 5.ª ed. Addison-Wesley. — Fundamentos de sistemas distribuídos e tolerância a falhas.